

【百战程序员】项目简介



项目概述

本项目是一个**健身房管理系统**，采用**前后端分离**架构：前端负责页面与交互，后端提供 REST API、会话与数据持久化。用户分为**管理员与会员**：管理员维护会员、员工、器材、课程与报名订单等；会员可查看/修改个人信息、报名课程，并使用内置 **DeepSeek** 聊天功能获取训练与饮食建议。

目录结构

- **前端**：Vue3 + Vite + TypeScript + Element Plus
- **后端**：Spring Boot + MyBatis + MySQL+deepseek 聊天接入

【百战程序员】创建框架



pom.xml

```
1 <?xml version="1.0" encoding="UTF-8"?>
2 <project
  xmlns="http://maven.apache.org/POM/4.0.0"
  xmlns:xsi="http://www.w3.org/2001/XMLSchema-
    instance"
3
  xsi:schemaLocation="http://maven.apache.org/
    POM/4.0.0
    https://maven.apache.org/xsd/maven-
    4.0.0.xsd">
4     <modelVersion>4.0.0</modelVersion>
5     <parent>
6
    <groupId>org.springframework.boot</groupId>
```

```
7         <artifactId>spring-boot-starter-  
parent</artifactId>  
8         <version>3.5.11</version>  
9         <relativePath/>  
10    </parent>  
11    <groupId>com.gym</groupId>  
12    <artifactId>gym-management-  
system</artifactId>  
13    <version>0.0.1-SNAPSHOT</version>  
14    <name>gym-management-system</name>  
15    <description>gym-management-  
system</description>  
16    <properties>  
17        <java.version>17</java.version>  
18  
19        <mysql.version>8.0.25</mysql.version>  
20        <mybatis-spring-boot-  
starter.version>3.0.4</mybatis-spring-boot-  
starter.version>  
21    </properties>  
22    <dependencies>  
23        <dependency>  
24            <groupId>org.springframework.boot</groupId>  
25            <artifactId>spring-boot-starter-  
web</artifactId>  
26            </dependency>  
27            <dependency>  
28                <groupId>org.mybatis.spring.boot</groupId>  
29                <artifactId>mybatis-spring-boot-  
starter</artifactId>
```

```
29         <version>${mybatis-spring-boot-
starter.version}</version>
30     </dependency>
31
32     <dependency>
33         <groupId>mysql</groupId>
34         <artifactId>mysql-connector-
java</artifactId>
35         <version>${mysql.version}
</version>
36         <scope>runtime</scope>
37     </dependency>
38     <dependency>
39
40     <groupId>org.projectlombok</groupId>
41         <artifactId>lombok</artifactId>
42         <optional>true</optional>
43     </dependency>
44 </dependencies>
45 <build>
46     <plugins>
47         <plugin>
48
49         <groupId>org.springframework.boot</groupId>
50         <artifactId>spring-boot-
maven-plugin</artifactId>
51     </plugin>
52 </plugins>
53 </build>
54 </project>
```

启动类

```
1 package com.gym;  
2  
3 import  
  org.mybatis.spring.annotation.MapperScan;  
4 import  
  org.springframework.boot.SpringApplication;  
5 import  
  org.springframework.boot.autoconfigure.Sprin  
    gBootApplication;  
6  
7 @SpringBootApplication  
8 @MapperScan("com.gym.mapper")  
9 public class GymManagementSystemApplication  
10 {  
11     public static void main(String[] args) {  
12  
13         SpringApplication.run(GymManagementSystemApp  
14             lication.class, args);  
15     }  
16 }
```

【百战程序员】数据库与 application 配置



application.yml

```
1  spring:
2    datasource:
3      url:
4        jdbc:mysql://localhost:3306/gym_management_s
5        ystem
6      username: root
7      password: 123456
8      driver-class-name:
9        com.mysql.cj.jdbc.Driver

10  mybatis:
```

```
10 mapper-locations: classpath:mapper/*.xml
11 configuration:
12     map-underscore-to-camel-case: true
13     type-aliases-package: com.gym.pojo
14
15 # DeepSeek 聊天配置（用于 /api/chat/query 调用）
16 deepseek:
17     api:
18         key: "your-deepseek-api-key"
19         url:
20             "https://api.deepseek.com/v1/chat/completions"
21     model: "deepseek-chat"
```

【百战程序员】登录功能



调用关系

层级	类 / 文件
Controller	<code>com.gym.controller.ApiLoginController</code>
Service	<code>com.gym.service.AdminService</code> → <code>com.gym.service.impl.AdminServiceImpl</code>
Mapper	<code>com.gym.mapper.AdminMapper</code>
SQL	<code>backend/src/main/resources/mapper/AdminMapper.xml</code>
实体	<code>com.gym.pojo.Admin</code>

实体 `Admin`

文件: `com/gym/pojo/Admin.java`


```
1 package com.gym.pojo;
2
3 import lombok.Data;
4
5 @Data
6 public class Admin {
7
8     private Integer adminAccount;
9     private String adminPassword;
10 }
```

Mapper 接口

文件: `com/gym/mapper/AdminMapper.java`

```
1 @Mapper
2 public interface AdminMapper {
3
4     Admin selectByAccountAndPassword(Admin
5     admin);
6 }
```

Mapper XML

文件: `src/main/resources/mapper/AdminMapper.xml`

```
1 <?xml version="1.0" encoding="UTF-8" ?>
2 <!DOCTYPE mapper
3     PUBLIC "-//mybatis.org//DTD Mapper
4     3.0//EN"
5     "http://mybatis.org/dtd/mybatis-3-
6     mapper.dtd">
7 <mapper
8     namespace="com.gym.mapper.AdminMapper">
9
10     <select id="selectByAccountAndPassword"
11         resultType="admin">
12         SELECT *
13         FROM admin
14         WHERE admin_account = #
15             {adminAccount}
16             AND admin_password = #
17             {adminPassword}
18     </select>
19
20 </mapper>
```

Service 接口

文件: `com/gym/service/AdminService.java`

```
1 package com.gym.service;
2
3 import com.gym.pojo.Admin;
4
5 public interface AdminService {
6
7     Admin adminLogin(Admin admin);
8 }
```

Service 实现

文件: `com/gym/service/impl/AdminServiceImpl.java`

```
1 @Service
2 public class AdminServiceImpl implements
   AdminService {
3
4     @Autowired
5     private AdminMapper adminMapper;
6
7     @Override
8     public Admin adminLogin(Admin admin) {
9         return
   adminMapper.selectByAccountAndPassword(admin
   );
10    }
11 }
```

Controller

文件: `com/gym/controller/ApiLoginController.java`

```
1  @RestController
2  @RequestMapping("/api")
3  public class ApiLoginController {
4
5      @PostMapping("/adminLogin")
6      public ResponseEntity<Map<String,
7      Object>> adminLogin(Admin admin, HttpSession
8      session) {
9          Admin loggedIn =
10         adminService.adminLogin(admin);
11         if (loggedIn == null) {
12             return unauthorized("账号或密码有
13             误");
14         }
15         // 保存信息
16         putAdminMainDataInSession(session,
17         loggedIn);
18         return
19         ResponseEntity.ok(singleSuccess());
20     }
21
22     private static Map<String, Object>
23     singleSuccess() {
24         Map<String, Object> m = new
25         HashMap<>(2);
26         m.put("success", true);
27     }
28 }
```

```
18         return m;
19     }
20
21     private static
    ResponseEntity<Map<String, Object>>
    unauthorized(String message) {
22         Map<String, Object> m = new
    HashMap<>(4);
23         m.put("success", false);
24         m.put("message", message);
25         return
    ResponseEntity.status(HttpStatus.UNAUTHORIZE
    D).body(m);
26     }
27 }
```

postman测试

- URL: POST http://localhost:8080/api/adminLogin
- Content-Type: application/x-www-form-urlencoded
- 参数示例
 - adminAccount:1001
 - adminPassword:123456

【百战程序员】Session 数据



在账号密码校验通过之后，将：

- 1 **当前登录管理员**写入 Session
- 2 **主页用到的统计**：会员总数、员工总数、人员合计（会员+员工）、器材总数，写入 Session，供 **GET /api/toAdminMain** 等读取。

核心方法

文件： `com/gym/controller/ApiLoginController.java`

```
1 private static final String SESSION_ADMIN =  
  "admin";  
2  
3  
4 private final MemberService memberService;  
5 private final AdminService adminService;  
6 private final EmployeeService  
  employeeService;  
7 private final EquipmentService  
  equipmentService;  
8  
9 public ApiLoginController(  
10     MemberService memberService,  
11     AdminService adminService,  
12     EmployeeService employeeService,  
13     EquipmentService equipmentService) {  
14     this.memberService = memberService;  
15     this.adminService = adminService;  
16     this.employeeService = employeeService;  
17     this.equipmentService =  
equipmentService;  
18 }  
19  
20  
21 private void  
  putAdminMainDataInSession(HttpSession  
  session, Admin admin) {  
22     session.setAttribute(SESSION_ADMIN,  
admin);  
23     Integer memberTotal =  
memberService.selectTotalCount();
```

```

24     Integer employeeTotal =
        employeeService.selectTotalCount();
25     Integer humanTotal = memberTotal +
        employeeTotal;
26     Integer equipmentTotal =
        equipmentService.selectTotalCount();
27     session.setAttribute("memberTotal",
        memberTotal);
28     session.setAttribute("employeeTotal",
        employeeTotal);
29     session.setAttribute("humanTotal",
        humanTotal);
30     session.setAttribute("equipmentTotal",
        equipmentTotal);
31 }

```

Session 键说明

Session 键	含义
admin (SESSION_ADMIN)	当前登录管理员对象
memberTotal	会员表行数
employeeTotal	员工表行数
humanTotal	memberTotal + employeeTotal
equipmentTotal	器材表行数

对象示例

```
1  @Data
2  public class Member {
3
4      private Integer memberAccount;
5      private String memberPassword;
6      private String memberName;
7      private String memberGender;
8      private Integer memberAge;
9      private Integer memberHeight;
10     private Integer memberWeight;
11     private Long memberPhone;
12     private String cardTime;
13     private Integer cardClass;
14     private Integer cardNextClass;
15 }
```

```
1  @Data
2  @NoArgsConstructor
3  @AllArgsConstructor
4  public class Employee {
5
6      private Integer employeeAccount;
7      private String employeeName;
8      private String employeeGender;
9      private Integer employeeAge;
10     private String entryTime;
11     private String staff;
12     private String employeeMessage;
13 }
```

```
1 @Data
2 @NoArgsConstructor
3 @AllArgsConstructor
4 public class Equipment {
5
6     private Integer equipmentId;
7     private String equipmentName;
8     private String equipmentLocation;
9     private String equipmentStatus;
10    private String equipmentMessage;
11 }
```

会员总数:

Mapper — `com/gym/mapper/MemberMapper.java`

```
1 Integer selectTotalCount();
```

XML — `resources/mapper/MemberMapper.xml`

```
1 <select id="selectTotalCount"
2     resultType="java.lang.Integer">
3     SELECT count(*)
4     FROM member
5 </select>
```

接口 — `com/gym/service/MemberService.java`

```
1 Integer selectTotalCount();
```

实现 — `com/gym/service/impl/MemberServiceImpl.java`

```
1 @Override
2 public Integer selectTotalCount() {
3     return memberMapper.selectTotalCount();
4 }
```

员工总数:

Mapper — `com/gym/mapper/EmployeeMapper.java`

```
1 Integer selectTotalCount();
```

XML — `resources/mapper/EmployeeMapper.xml`

```
1 <select id="selectTotalCount"
2     resultType="java.lang.Integer">
3     SELECT count(*)
4     FROM employee
5 </select>
```

接口 — `com/gym/service/EmployeeService.java`

```
1 Integer selectTotalCount();
```

实现 — `com/gym/service/impl/EmployeeServiceImpl.java`

```
1 @Override
2 public Integer selectTotalCount() {
3     return employeeMapper.selectTotalCount();
4 }
```

器材总数:

Mapper — `com/gym/mapper/EquipmentMapper.java`

```
1 Integer selectTotalCount();
```

XML — `resources/mapper/EquipmentMapper.xml`

```
1 <select id="selectTotalCount"
2     resultType="java.lang.Integer">
3     SELECT count(*)
4     FROM equipment
5 </select>
```

接口 — `com/gym/service/EquipmentService.java`

```
1 Integer selectTotalCount();
```

实现 —

`com/gym/service/impl/EquipmentServiceImpl.java`

```
1 @Override
2 public Integer selectTotalCount() {
3     return
4     equipmentMapper.selectTotalCount();
5 }
```

与 **adminLogin** 的衔接

文件: **com/gym/controller/ApiLoginController.java**

```
1 @PostMapping("/adminLogin")
2 public ResponseEntity<Map<String, Object>>
3     adminLogin(Admin admin, HttpSession session)
4     {
5         Admin loggedIn =
6         adminService.adminLogin(admin);
7         if (loggedIn == null) {
8             return unauthorized("账号或密码有误");
9         }
10        putAdminMainDataInSession(session,
11        loggedIn);
12        return
13        ResponseEntity.ok(singleSuccess());
14    }
```

【百战程序员】 员工查询



`com/gym/mapper/EmployeeMapper.java`

```
1 package com.gym.mapper;  
2  
3 import com.gym.pojo.Employee;  
4 import org.apache.ibatis.annotations.Mapper;  
5  
6 import java.util.List;  
7  
8 @Mapper  
9 public interface EmployeeMapper {  
10  
11     List<Employee> findAll();  
12 }
```

Mapper XML

resources/mapper/EmployeeMapper.xml

```
1 <select id="findAll" resultType="employee">  
2     SELECT *  
3     FROM employee  
4 </select>
```

Service

com/gym/service/EmployeeService.java

```
1 public interface EmployeeService {  
2  
3     List<Employee> findAll();  
4 }
```

backend/src/main/java/com/gym/service/impl/EmployeeServiceImpl.java

```
1  @Service
2  public class EmployeeServiceImpl implements
   EmployeeService {
3
4      @Autowired
5      private EmployeeMapper employeeMapper;
6
7      @Override
8      public List<Employee> findAll() {
9          return employeeMapper.findAll();
10     }
11 }
```

Controller

com/gym/controller/ApiEmployeeController.java

```
1  package com.gym.controller;
2
3  @RestController
4  @RequestMapping("/api/employee")
5  public class ApiEmployeeController {
6
7      @Autowired
8      private EmployeeService employeeService;
9
10     @GetMapping("/selEmployee")
```



```
11     public Map<String, Object>
selectEmployee() {
12         List<Employee> employeeList =
employeeService.findAll();
13         Map<String, Object> resp = new
HashMap<>();
14         resp.put("success", true);
15         resp.put("employeeList",
employeeList);
16         return resp;
17     }
18 }
```

【百战程序员】新增员工



Mapper

[com/gym/mapper/EmployeeMapper.java](#)

```
1 package com.gym.mapper;  
2  
3 import com.gym.pojo.Employee;  
4 import org.apache.ibatis.annotations.Mapper;  
5  
6 @Mapper  
7 public interface EmployeeMapper {  
8  
9     Boolean insertEmployee(Employee  
10    employee);  
11 }
```

Mapper XML

[resources/mapper/EmployeeMapper.xml](#)

```
1 <insert id="insertEmployee"
  parameterType="employee">
2     INSERT INTO employee (employee_account,
  employee_name, employee_gender,
3         employee_age,
  entry_time, staff, employee_message)
4     VALUES ({employeeAccount}, #
  {employeeName}, #{employeeGender},
5         #{employeeAge}, #{entryTime}, #
  {staff}, #{employeeMessage})
6 </insert>
```

Service

com/gym/service/EmployeeService.java

```
1 package com.gym.service;
2
3 import com.gym.pojo.Employee;
4
5 public interface EmployeeService {
6
7     Boolean insertEmployee(Employee
  employee);
8 }
```

com/gym/service/impl/EmployeeServiceImpl.java

```

1  @Service
2  public class EmployeeServiceImpl implements
   EmployeeService {
3
4      @Autowired
5      private EmployeeMapper employeeMapper;
6
7      @Override
8      public Boolean insertEmployee(Employee
   employee) {
9          return
   employeeMapper.insertEmployee(employee);
10     }
11 }

```

Controller

com/gym/controller/ApiEmployeeController.java

```

1  @RestController
2  @RequestMapping("/api/employee")
3  public class ApiEmployeeController {
4
5      @Autowired
6      private EmployeeService employeeService;
7
8      @PostMapping("/addEmployee")
9      public ResponseEntity<Map<String,
   Object>> addEmployee(Employee employee) {
10         Random random = new Random();

```

```
11         String account1 = "1010";
12         for (int i = 0; i < 5; i++) {
13             account1 += random.nextInt(10);
14         }
15         Integer account =
Integer.parseInt(account1);
16
17         Date date = new Date();
18         SimpleDateFormat simpleDateFormat =
new SimpleDateFormat("yyyy-MM-dd");
19         String nowDay =
simpleDateFormat.format(date);
20
21
22         employee.setEmployeeAccount(account);
23         employee.setEntryTime(nowDay);
24
25         employeeService.insertEmployee(employee);
26
27         Map<String, Object> resp = new
HashMap<>();
28         resp.put("success", true);
29         return ResponseEntity.ok(resp);
30     }
```

【百战程序员】删除员工



Mapper

[com/gym/mapper/EmployeeMapper.java](#)

```
1 package com.gym.mapper;  
2  
3 import org.apache.ibatis.annotations.Mapper;  
4  
5 @Mapper  
6 public interface EmployeeMapper {  
7  
8     Boolean deleteByEmployeeAccount(Integer  
employeeAccount);  
9 }
```

Mapper XML

resources/mapper/EmployeeMapper.xml

```
1 <delete id="deleteByEmployeeAccount"  
  parameterType="java.lang.Integer">  
2     DELETE  
3     FROM employee  
4     WHERE employee_account = #  
      {employeeAccount}  
5 </delete>
```

com/gym/service/EmployeeService.java

```
1 package com.gym.service;  
2  
3 public interface EmployeeService {  
4  
5     Boolean deleteByEmployeeAccount(Integer  
      employeeAccount);  
6 }
```

com/gym/service/impl/EmployeeServiceImpl.java

```
1 @Service
2 public class EmployeeServiceImpl implements
  EmployeeService {
3
4     @Autowired
5     private EmployeeMapper employeeMapper;
6
7     @Override
8     public Boolean
  deleteByEmployeeAccount(Integer
  employeeAccount) {
9         return
  employeeMapper.deleteByEmployeeAccount(emplo
  yeeAccount);
10    }
11 }
```

Controller

[com/gym/controller/ApiEmployeeController.java](#)

```
1 @RestController
2 @RequestMapping("/api/employee")
3 public class ApiEmployeeController {
4
5     @Autowired
6     private EmployeeService employeeService;
7
8     @PostMapping("/delEmployee")
```



```
9      public ResponseEntity<Map<String,  
Object>> deleteEmployee(Integer  
employeeAccount) {  
10  
    employeeService.deleteByEmployeeAccount(emp  
oyeeAccount);  
11      Map<String, Object> resp = new  
HashMap<>();  
12      resp.put("success", true);  
13      return ResponseEntity.ok(resp);  
14  }  
15 }
```

【百战程序员】修改员工



Mapper

[com/gym/mapper/EmployeeMapper.java](#)

```
1 package com.gym.mapper;  
2  
3 import com.gym.pojo.Employee;  
4 import org.apache.ibatis.annotations.Mapper;  
5  
6 @Mapper  
7 public interface EmployeeMapper {  
8  
9     Boolean  
    updateMemberByEmployeeAccount(Employee  
    employee);  
10 }
```

Mapper XML

[resources/mapper/EmployeeMapper.xml](#)

```
1 <update id="updateMemberByEmployeeAccount"
  parameterType="employee">
2     UPDATE employee
3     SET employee_account = #
    {employeeAccount},
4         employee_name      = #{employeeName},
5         employee_gender    = #
    {employeeGender},
6         employee_age       = #{employeeAge},
7         staff              = #{staff},
8         employee_message=#{employeeMessage}
9     WHERE employee_account = #
    {employeeAccount}
10 </update>
```

Service

com/gym/service/EmployeeService.java

```
1 package com.gym.service;
2
3 import com.gym.pojo.Employee;
4
5 public interface EmployeeService {
6
7     Boolean
    updateMemberByEmployeeAccount(Employee
    employee);
8 }
```

com/gym/service/impl/EmployeeServiceImpl.java

```
1 @Service
2 public class EmployeeServiceImpl implements
    EmployeeService {
3
4     @Autowired
5     private EmployeeMapper employeeMapper;
6
7     @Override
8     public Boolean
    updateMemberByEmployeeAccount(Employee
    employee) {
9         return
    employeeMapper.updateMemberByEmployeeAccount
    (employee);
10    }
11 }
```

Controller

com/gym/controller/ApiEmployeeController.java

```
1  @RestController
2  @RequestMapping("/api/employee")
3  public class ApiEmployeeController {
4
5      @Autowired
6      private EmployeeService employeeService;
7
8      @PostMapping("/updateEmployee")
9      public ResponseEntity<Map<String,
10 Object>> updateEmployee(Employee employee) {
11
12      employeeService.updateMemberByEmployeeAccount(employee);
13      Map<String, Object> resp = new
14      HashMap<>();
15      resp.put("success", true);
16      return ResponseEntity.ok(resp);
17  }
18 }
```

跳转修改列表

```

1 @GetMapping("/toUpdateEmployee")
2 public Map<String, Object>
   toUpdateEmployee(Integer employeeAccount) {
3     List<Employee> employeeList =
   employeeService.selectByEmployeeAccount(emplo
   yeeAccount);
4     Map<String, Object> resp = new HashMap<>
   ();
5     resp.put("success", true);
6     resp.put("employeeList", employeeList);
7     return resp;
8 }

```

Service

```

1 List<Employee>
   selectByEmployeeAccount(Integer
   employeeAccount);

```

```

1 @Override
2 public List<Employee>
   selectByEmployeeAccount(Integer
   employeeAccount) {
3     return
   employeeMapper.selectByEmployeeAccount(employ
   eeAccount);
4 }

```

Mapper

```
1 List<Employee>
  selectByEmployeeAccount(Integer
    employeeAccount);
```

```
1 <select id="selectByEmployeeAccount"
  parameterType="java.lang.Integer"
  resultType="employee">
2     SELECT *
3     FROM employee
4     WHERE employee_account = #
    {employeeAccount}
5 </select>
```

【百战程序员】DeepSeek聊天功能



DeepSeek获取接口网址:

https://platform.deepseek.com/sign_in

com/gym/service/ChatService.java

```
1 package com.gym.service;
2
3 public interface ChatService {
4
5     /**
6      * @param content 用户输入
7      * @param model    DeepSeek 模型，允许为空
8      *                (为空则使用默认模型)
9      * @return AI 回复文本
10      */
11     String queryChat(String content, String
12                       model);
13 }
```



```
1 @Service
2 public class ChatServiceImpl implements
   ChatService {
3
4     @Value("${deepseek.api.key}")
5     private String apiKeyFromConfig;
6
7
8     @Value("${deepseek.api.url:https://api.deepseek.com/v1/chat/completions}")
9     private String apiUrl;
10
11     @Value("${deepseek.model:deepseek-chat}")
12     private String defaultModel;
13
14     @Override
15     public String queryChat(String content,
16                             String model) {
17         String userContent = content == null
18             ? "" : content.trim();
19         if (userContent.isEmpty()) {
20             return "我还没收到你的问题，可以再发一次吗? ";
21         }
22
23         String resolvedModel = (model == null || model.trim().isEmpty()) ?
24             defaultModel : model.trim();
25         String apiKey = apiKeyFromConfig;
```

```
22         if (apiKey == null ||
apiKey.trim().isEmpty()) {
23             apiKey =
System.getenv("DEEPSEEK_API_KEY");
24         }
25         if (apiKey == null ||
apiKey.trim().isEmpty()) {
26             throw new
IllegalStateException("未配置 DeepSeek API
Key");
27         }
28
29         try {
30             ObjectMapper objectMapper = new
ObjectMapper();
31
32             Map<String, Object> payload =
new HashMap<>();
33             payload.put("model",
resolvedModel);
34             payload.put("stream", false);
35
36             List<Map<String, String>>
messages = new ArrayList<>();
37             messages.add(new HashMap<String,
String>() {{
38                 put("role", "system");
39                 put("content", "你是一个健身房
训练与饮食建议助手。回答要具体、可执行，并给出需要的
注意事项。")
```

```
40         + "如果用户消息中包含“系  
系统补充信息”里列出的会员课程，请先用简短的中文总结  
出“该会员报名了哪些课程”，"  
41         + "然后再结合这些课程安  
排来回答用户的后续问题。");  
42     });  
43     messages.add(new HashMap<String,  
String>() {{  
44         put("role", "user");  
45         put("content", userContent);  
46     }});  
47     payload.put("messages",  
messages);  
48  
49     String requestBody =  
objectMapper.writeValueAsString(payload);  
50  
51     HttpURLConnection connection =  
(HttpURLConnection) new  
URL(apiUrl).openConnection();  
52  
connection.setRequestMethod("POST");  
53     connection.setDoOutput(true);  
54  
connection.setRequestProperty("Authorization  
", "Bearer " + apiKey);  
55  
connection.setRequestProperty("Content-  
Type", "application/json; charset=UTF-8");  
56  
57     OutputStream os =  
connection.getOutputStream();
```

```
58      os.write(requestBody.getBytes(StandardCharsets.UTF_8));
59          os.flush();
60          os.close();
61
62          int statusCode =
connection.getResponseCode();
63          InputStream is = statusCode >=
200 && statusCode < 300 ?
connection.getInputStream() :
connection.getErrorStream();
64          String responseBody =
readAll(is);
65
66          if (statusCode < 200 ||
statusCode >= 300) {
67              throw new
RuntimeException("DeepSeek API 请求失败,
status=" + statusCode + ", body=" +
responseBody);
68          }
69
70          JsonNode root =
objectMapper.readTree(responseBody);
71          JsonNode choices =
root.path("choices");
72          if (choices.isArray() &&
choices.size() > 0) {
73              JsonNode message =
choices.get(0).path("message");
```

```
74         String reply =
message.path("content").asText(null);
75         if (reply != null &&
!reply.trim().isEmpty()) {
76             return reply;
77         }
78     }
79
80     throw new
RuntimeException("DeepSeek 返回无法解析:
choices[0].message.content 为空");
81     } catch (IOException e) {
82         throw new
RuntimeException("DeepSeek 调用失败: " +
e.getMessage(), e);
83     }
84 }
85
86 private String readAll(InputStream is)
throws IOException {
87     if (is == null) return "";
88     StringBuilder sb = new
StringBuilder();
89     BufferedReader br = new
BufferedReader(new InputStreamReader(is,
StandardCharsets.UTF_8));
90     String line;
91     while ((line = br.readLine()) !=
null) {
92         sb.append(line);
93     }
94     br.close();
```

```
95         return sb.toString();  
96     }  
97 }
```

【百战程序员】会员课程信息



[com/gym/controller/ApiChatController.java](#)

```
1  @RestController  
2  @RequestMapping("/api/chat")  
3  public class ApiChatController {  
4  
5      @Autowired
```

```
6     private ChatService chatService;
7
8     @Autowired
9     private ClassOrderService
classOrderService;
10
11     @PostMapping("/query")
12     public Map<String, Object> query(
13         @RequestParam("content") String
content,
14         @RequestParam(required = false)
String model,
15         HttpSession session
16     ) {
17         Map<String, Object> resp = new
HashMap<>();
18         try {
19             Member member = (Member)
session.getAttribute("user");
20             String enhancedContent =
buildContentwithMemberClasses(content,
member);
21
22             String reply =
chatService.queryChat(enhancedContent,
model);
23             resp.put("success", true);
24             resp.put("reply", reply);
25         } catch (Exception e) {
26             resp.put("success", false);
27             resp.put("message",
e.getMessage());
```

```
28         }
29         return resp;
30     }
31
32     private String
buildContentWithMemberClasses(String
originalContent, Member member) {
33         if (member == null ||
member.getMemberAccount() == null) {
34             return originalContent;
35         }
36
37         List<ClassOrder> classOrders =
classOrderService.selectClassOrderByMemberAc
count(member.getMemberAccount());
38         if (classOrders == null ||
classOrders.isEmpty()) {
39             return originalContent;
40         }
41
42         String classInfo =
classOrders.stream()
43             .map(order -> String.format(
44                 "课程名称: %s, 教练:
%s, 上课时间: %s",
45                 order.getClassName(),
46                 order.getCoach(),
47                 order.getClassBegin()
48             ))
```



```

49         .collect(Collectors.joining("; "));
50
51         String memberInfo =
String.format("会员姓名: %s, 会员账号: %s",
52             member.getMemberName(),
member.getMemberAccount());
53
54         StringBuilder sb = new
StringBuilder();
55         sb.append(originalContent == null ?
"" : originalContent.trim());
56         sb.append("\n\n");
57         sb.append("【系统补充信息】下面是该会员当
前已报名的课程信息，请先根据这些信息，明确地告诉用
户“我报名了哪些课程”，然后再结合课程安排回答后续问
题: \n");
58         sb.append(memberInfo).append("\n");
59         sb.append(classInfo);
60         return sb.toString();
61     }
62 }

```

【百战程序员】会话鉴权



问题：在未登录的情况下输入网址

<http://localhost:8080/api/employee/selEmployee>

<http://localhost:5173/member/selMember>

统一会话鉴权拦截器

`com/gym/security/SessionAuthInterceptor.java`

```
1 @Component
2 public class SessionAuthInterceptor
3     implements HandlerInterceptor {
4
5     private final ObjectMapper objectMapper;
6
7     public
8     SessionAuthInterceptor(ObjectMapper
9     objectMapper) {
10         this.objectMapper = objectMapper;
11     }
12 }
```

```
10     private boolean isPublicApi(String uri)
11     {
12         return uri == null
13             ||
14         uri.startsWith("/api/adminLogin")
15             ||
16         uri.startsWith("/api/userLogin")
17             ||
18         uri.startsWith("/api/logout");
19     }
20
21     private boolean isUserApi(String uri) {
22         return uri != null && (
23
24         uri.equals("/api/toUserMain")
25             ||
26         uri.startsWith("/api/user/")
27             ||
28         uri.startsWith("/api/chat/")
29             );
30     }
31
32     @Override
33     public boolean
34     preHandle(HttpServletRequest request,
35     HttpServletResponse response, Object
36     handler) throws Exception {
37         if
38         ("OPTIONS".equalsIgnoreCase(request.getMethod())) {
39
40             return true;
41         }
42     }
```

```
30
31     String uri =
request.getRequestURI();
32     if (uri == null ||
!uri.startsWith("/api/")) {
33         return true;
34     }
35
36     if (isPublicApi(uri)) {
37         return true;
38     }
39
40     boolean requireUser =
isUserApi(uri);
41     String requiredSessionKey =
requireUser ? "user" : "admin";
42
43     HttpSession session =
request.getSession(false);
44     if (session != null &&
session.getAttribute(requiredSessionKey) !=
null) {
45         return true;
46     }
47
48     response.setStatus(HttpServletResponse.SC_UN
AUTHORIZED);
49
response.setCharacterEncoding(StandardCharse
ts.UTF_8.name());
```

```

50     response.setContentType(MediaType.APPLICATION_JSON_VALUE);
51
52     Map<String, Object> resp = new
HashMap<>();
53     resp.put("success", false);
54     resp.put("message", "未登录");
55
56     response.getWriter().write(objectMapper.writeValueAsString(resp));
57     return false;
58 }

```

注册拦截器

com/gym/config/WebMvcConfig.java

```

1  @Configuration
2  public class webMvcConfig implements
webMvcConfigurer {
3
4      private final SessionAuthInterceptor
sessionAuthInterceptor;
5
6      public
webMvcConfig(SessionAuthInterceptor
sessionAuthInterceptor) {
7          this.sessionAuthInterceptor =
sessionAuthInterceptor;
8      }

```

```
9
10     @Override
11     public void addCorsMappings(CorsRegistry
registry) {
12         registry.addMapping("/**")
13
14         .allowedOrigins("http://localhost:5173")
15         .allowedMethods("GET",
"POST", "PUT", "PATCH", "DELETE", "OPTIONS")
16         .allowedHeaders("*")
17         .allowCredentials(true);
18     }
19
20     @Override
21     public void
addInterceptors(InterceptorRegistry
registry) {
22         registry.addInterceptor(sessionAuthIntercept
or)
23         .addPathPatterns("/api/**")
24         .excludePathPatterns(
25             "/api/adminLogin",
26             "/api/userLogin",
27             "/api/logout"
28         );
29     }
```

